

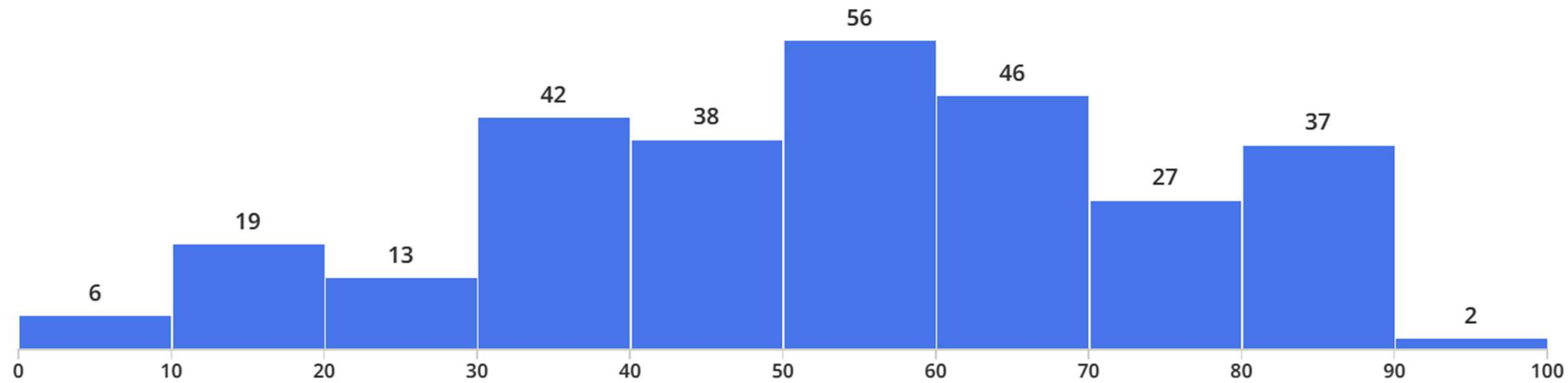
EECS 482

Introduction to operating systems

Post midterm

Peter Chen
University of Michigan

Midterm results



Minimum

2.0

Median

53.0

Maximum

91.0

Mean

52.85

Std Dev [?](#)

20.85

1 Analyzing test case behavior on different thread libraries (24 points)

Your task is to analyze a test case's behavior on two different thread libraries.

The first thread library is the Project 2 thread library, which runs threads in FIFO order.

The second thread library is the same as the Project 2 thread library, **except it runs threads in order of age**. Among runnable threads, threads that are younger (i.e., were created later) should **always** run before threads that are older (i.e., were created earlier). **Hint: consider how this policy affects thread creation.**

The following questions ask you to count the maximum possible number of living (i.e., ready or running) threads in the test case below. When counting threads, only consider threads created by the test case (i.e., the threads running parent, child, or grandchild). The test case is running on a uniprocessor. Interrupt generation will be specified in each question.

- Key concepts
 - Analyzing concurrent programs
 - Thread scheduling
 - Interrupts

1.1

What is the maximum possible number of living (i.e., ready or running) threads in this test case if it uses a Project 2 thread library and does **not** generate interrupts? Justify your answer.

- Execution trace

Parent (1 thread)

Parent creates 5 children (6 threads), then ends (5 threads)

Child #1 creates 10 grandchildren (15 threads), then ends (14 threads)

Child #2 creates 10 grandchildren (24 threads), then ends (23 threads)

Child #3 creates 10 grandchildren (33 threads), then ends (32 threads)

Child #4 creates 10 grandchildren (42 threads), then ends (41 threads)

Child #5 creates 10 grandchildren (51 threads), then ends (50 threads)

...

1.2

What is the maximum possible number of living (i.e., ready or running) threads in this test case above if it uses a Project 2 thread library and **does** generate interrupts? Justify your answer.

- Interrupts can occur (or not occur) at any time
- The possibility of interrupts can only **increase** the maximum possible number of living threads (since no interrupts is also a possibility)
- What if each thread were interrupted just before it ends (and keeps getting interrupted)?
- Maximum number of living threads
 - = total threads in the program
 - = 1 parent + 5 children + 50 grandchildren (56 threads)

The second thread library is the same as the Project 2 thread library, **except it runs threads in order of age**. Among runnable threads, threads that are younger (i.e., were created later) should **always** run before threads that are older (i.e., were created earlier). **Hint: consider how this policy affects thread creation.**

1.3

What is the maximum possible number of living (i.e., ready or running) threads in this test case if it uses the thread library that runs younger threads before older threads, and does **not** generate interrupts? Justify your answer.

- Which thread should be running?
- How does this affect thread creation?
- Execution trace
 - Parent (1 thread)
 - Parent creates child #1 (2 threads)
 - Child #1 preempts parent (2 threads)
 - Child #1 creates grandchild (3 threads)
 - Grandchild preempts child #1 (3 threads), then ends (2 threads)
 - Child #1 creates grandchild (3 threads)
 - Grandchild preempts child #1 (3 threads), then ends (2 threads)
 - ...
- How many siblings can be alive at the same time?

1.4

What is the maximum possible number of living (i.e., ready or running) threads in this test case if it uses the thread library that runs younger threads before older threads, and **does** generate interrupts? Justify your answer.

- Do interrupts affect which thread should run under the youngest-first policy?

2 Implementing join (monitor programming) (38 points)

You are given a modified Project 1 thread library that does not support `thread::join`. Your task is to implement a new class `jthread` that supports `join`, using only this modified Project 1 thread library.

Multi-threaded programs should be able to use `jthread` as a drop-in replacement for the `thread` class; programs should not need to make any other changes to use `jthread`.

This is a user-level programming question. Use monitors for synchronization. You may use the following classes from Project 1: `mutex`, `cv`, `thread` (but **not** `thread::join`).

- Which type of problem is this (user-level or kernel-level)?
- Which lecture example is this most similar to?
- Key concepts
 - Creating threads
 - Monitor programming
 - Join
 - Thread execution lifetime vs. thread object lifetime
 - How to add functionality to someone else's function

Pattern for monitor programs

lock

while (condition) wait
<do stuff>
signal if needed

repeat as needed

unlock

Writing concurrent programs

- Think about threads independently and greedily
- List the shared data needed to solve the problem
 - Shared variable: `threadIsAlive`
- Assign a lock per group of shared data, and add lock/unlock around code that uses shared data
- List before-after conditions, and assign condition variable per waiting condition
 - `thread::join` must wait while the thread it's waiting for is alive

Where to store threadIsalive?

- Read by the thread that calls thread::join
 - Accessed by object of thread being joined with
- Written by the thread that is being joined with
 - Accessed by stream of execution of thread being joined with
- Thread object and thread execution have different lifetimes. Either can end first.
- What programming tool can help us manage a variable whose lifetime varies?

```
class jthread {  
    ...  
    shared_ptr<join_info> ji_ptr;  
};
```

```
struct join_info {  
    mutex m;  
    cv waiters;  
    bool threadIsAlive = true;  
};
```

```
struct wrapper_info {  
    thread_startfunc_t func;  
    uintptr_t arg;  
    shared_ptr<join_info> ji_ptr;  
};
```

```
jthread::jthread(thread_startfunc_t func, uintptr_t arg) {  
    ji_ptr = make_shared<join_info>();  
    thread::thread(  
        wrapper,  
        new wrapper_info(func, arg, ji_ptr));  
}
```

```
void wrapper(wrapper_info* w_ptr) {  
    w_ptr->func(w_ptr->arg);  
  
    w_ptr->ji_ptr->m.lock();  
    w_ptr->ji_ptr->threadIsAlive = false;  
    w_ptr->ji_ptr->waiters.broadcast();  
    w_ptr->ji_ptr->m.unlock();  
  
    delete w_ptr;  
}
```

```
void jthread::join() {  
    ji_ptr->m.lock();  
    while (ji_ptr->threadIsAlive) {  
        ji_ptr->waiters.wait(ji_ptr->m);  
    }  
    ji_ptr->m.unlock();  
}
```


Question 3.1B: Write a short, self-contained test program that calls `swapcontext` one or more times to show how it switches between contexts. You may use `getcontext`, `makecontext`, `setcontext`, and `swapcontext` (and normal C++ code). Use `cout` statements to show that `swapcontext` is working correctly.

```
ucontext_t A, B;
char stackA[STACK_SIZE], stackB[STACK_SIZE];
int main() {
    makecontext(&A, stackA, STACK_SIZE, funcA);
    makecontext(&B, stackB, STACK_SIZE, funcB);
    setcontext(&A);
}
void funcA() {
    cout << "A before swapcontext\n";
    swapcontext(&A, &B);
    cout << "A after swapcontext\n";
}
void funcB() {
    cout << "B before swapcontext\n";
    swapcontext(&B, &A);
    cout << "B after swapcontext\n";
}
```

3 Implementing swapcontext (38 points)

In this question, you will implement `swapcontext` using `getcontext`, `makecontext`, and `setcontext`. Here are the prototypes for these functions:

```
int getcontext(ucontext_t* ucp);  
void makecontext(ucontext_t* ucp, char* stack, unsigned int stack_size, void (*func)(), int argc, ...);  
int setcontext(ucontext_t* ucp);  
int swapcontext(ucontext_t* old_ucp, ucontext_t* new_ucp);
```

You may assume interrupts are disabled, and there is only one processor.

- Key concepts
 - Creating and using thread contexts
 - Execution tracing through context switches
 - Behavior of `getcontext`, `setcontext`, `swapcontext`
- What is stored in a thread context?
 - Program counter
 - Stack pointer (but not stack)
 - Registers

From Sep. 22 lecture

Switching threads

- `getcontext()` copies context from CPU to memory
- `setcontext()` copies context from memory to CPU
- Does the following code work?

Thread A

```
do stuff  
getcontext (saves state of thread A)  
setcontext to thread B  
do more stuff
```

Thread B

```
do stuff  
getcontext (saves state of thread B)  
setcontext to thread A
```

- After B calls `setcontext()`, where does A resume execution?
- Where do we want A to resume execution?
- Linux: `swapcontext()` correctly combines `getcontext()` and `setcontext()`

Question 3.3

Consider the following implementation of `swapcontext`:

```
int swapcontext_3_3 (ucontext_t* old_ucp, ucontext_t* new_ucp) {  
    getcontext(old_ucp);  
    setcontext(new_ucp);  
    return(0);  
}
```

- What's wrong with this implementation?
- `getcontext` returns twice
 - When **this thread** calls it
 - » Should call `setcontext` afterwards
 - When **another thread** swaps to it
 - » Should not call `setcontext` afterwards
 - How to do different things for these two situations?

Question 3.2

Consider the following implementation of `swapcontext`:

```
bool flag = false;

int swapcontext_3_2 (ucontext_t* old_ucp, ucontext_t* new_ucp) {
    getcontext(old_ucp);
    if (!flag) {
        flag = true;
        setcontext(new_ucp);
    }
    return(0);
}
```

- Does this call `setcontext` after calling `getcontext`?
- Does this call `setcontext` after another thread swaps to this thread?
- What's wrong with this implementation?

Question 3.4

Consider the following implementation of `swapcontext`:

```
bool flag;

int swapcontext_3_4 (ucontext_t* old_ucp, ucontext_t* new_ucp) {
    flag = false;
    getcontext(old_ucp);
    if (!flag) {
        flag = true;
        setcontext(new_ucp);
    }
    return(0);
}
```

- Does this call `setcontext` after calling `getcontext`?
- Does this call `setcontext` after another thread swaps to this thread?
- What ensures that `flag==true` after another thread swaps to this thread?

Question 3.5

Consider the following implementation of `swapcontext`:

```
int swapcontext_3_5 (ucontext_t* old_ucp, ucontext_t* new_ucp) {  
    bool flag = false;  
    getcontext(old_ucp);  
    if (!flag) {  
        flag = true;  
        setcontext(new_ucp);  
    }  
    return(0);  
}
```

- Is this correct?
- Where is flag stored?
- What does `getcontext/setcontext` save and restore?

Question 3.1A: Implement `swapcontext`. You may use `getcontext`, `makecontext`, and `setcontext` (and normal C++ code). You may not use `swapcontext` (since you're implementing it).

```
int swapcontext(struct ucontext_t* old_ucp,  
               struct ucontext_t* new_ucp) {  
    auto flag_ptr = make_unique<bool>(false);  
    getcontext(old_ucp);  
    if (! *flag_ptr) {  
        *flag_ptr = true;  
        setcontext(new_ucp);  
    }  
    return(0);  
}
```

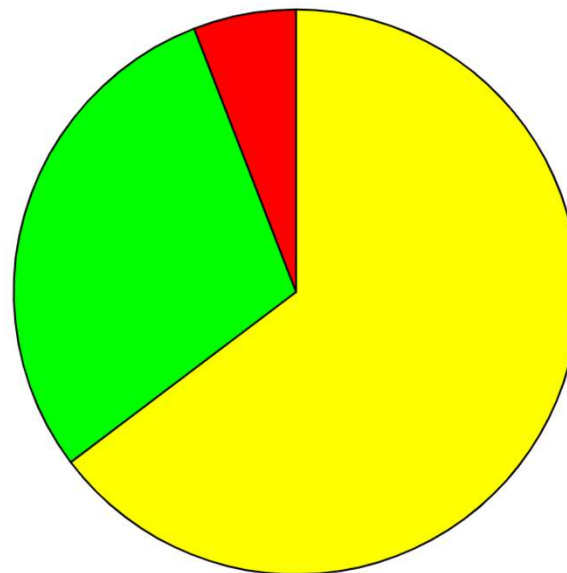

Question 3.1A: Implement `swapcontext`. You may use `getcontext`, `makecontext`, and `setcontext` (and normal C++ code). You may not use `swapcontext` (since you're implementing it).

```
map<ucontext_t*, bool> flag;
int swapcontext(struct ucontext_t* old_ucp,
               struct ucontext_t* new_ucp) {
    flag[old_ucp] = false;
    getcontext(old_ucp);
    if (! flag[old_ucp]) {
        flag[old_ucp] = true;
        setcontext(new_ucp);
    }
    flag.erase(old_ucp);
    return(0);
}
```

Regrade requests

- Regrades open October 23-29
- Explain why a question on your exam was graded incorrectly
- We will re-grade entire question (your score may go up or down)
- You may have one regrade request at a time. If your regrade is approved, you may submit another request

● same ● increase ● decrease



Progress at
you may

Overall grade status

- 45 points so far
- To extrapolate end-of-semester score:
 $P1*5 + P2*45 + \text{midterm}*50$
 ≈ 73 B / B+ boundary
 ≈ 65 B- / C+ boundary
 ≈ 57 C / C- boundary

These boundaries will change (probably increase) during the semester, esp. after the final exam.